

Implementación de un robot basado en Astromech usando ROS con recursos limitados

Universidad Nacional de Colombia
Sede Bogotá
Facultad de Ingeniería
Robótica y Control Servovisual
2027504

Camilo CAMPO P.
Ing. Mecatrónica
cecampop@unal.edu.co

Alejandro OJEDA O.
Ing. Mecatrónica
aojedao@unal.edu.co

Camilo TORRES M.
Ing. Mecatrónica
juctorresme@unal.edu.co

Resumen—El presente informe explora el proceso de diseño, implementación y optimización de un robot de tracción diferencial equipado con un sistema de control servovisual, implementado mediante el uso de la plataforma ROS, para la asistencia de personal técnico en el sector de la mecánica automotriz. La construcción del sistema físico se realiza mediante el uso de una tarjeta controladora Raspberry Pi 3B conectada con ROS Kinetic.

Palabras claves—astromech,

I. INTRODUCCIÓN

R2D2 es quizás el robot más popular de la ciencia ficción. Su fama no sólo se debe a la curiosa forma que tiene de comunicarse, ni a la versátil manera en la que interactúa con su entorno en las situaciones complicadas, sino también a su eficiente manera de desplazarse. R2D2 es un *astro-mech*, una clase de robot (en el universo ficticio de Star Wars) diseñado para brindar asistencia técnica a un piloto o a un mecánico en tareas de reparación, por ende su diseño está basado en garantizar una locomoción eficiente mientras se tiene como objetivo el seguimiento de un humano.

Mientras que en el sector de automatización para manufactura, la robótica ha alcanzado su cúspide, en lo que a robots asistenciales respecta, la investigación se centra alrededor de aplicaciones médicas en entornos de rehabilitación e investigación, además de aplicaciones de apoyo a emergencias. La robótica asistencial se ha enfrascado en diseñar soluciones basadas en mecanismos de movimiento bio-inspirados que representan una enorme complejidad. Solamente pudiendo alcanzar avances notables con el uso de técnicas de inteligencia artificial complejas y costosas a nivel computacional (como Deep Learning).

Sin embargo se proyecta que surjan nuevas aplicaciones conforme se den avances notables en el campo de la inteligencia artificial y se supere el escepticismo general con respecto a convivir con robots móviles autónomos, y que estas máquinas puedan ser aplicadas a nuevas industrias [1]. El uso de robots asistenciales en entornos de la industria automotriz tiene un antecedente en el proyecto ART de Audi [2] [3], en el que



Figura 1. Distintos modelos de *astromechs*, las características en común son el soporte tripodal y la tracción diferencial.

se implementaron plataformas *VGo* (plataformas de tracción diferencial operadas a control remoto por técnicos certificados) para diagnosticar y reparar automóviles. Han existido otros esfuerzos por implementar robots asistenciales con un enfoque de costo-efectividad [4], es aquí donde se plantea el uso de plataformas diferenciales



Figura 2. Audi Robotics Telepresence™, un robot operado por control remoto diseñado para el diagnóstico y reparación en entornos automotrices. Tomado de [2].

I-A. Modelo cinemático

El robot, como la mayoría de astromech, tendrá 3 llantas, dos traseras que proporcionarán el movimiento diferencial y una tercera de apoyo.

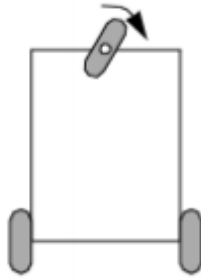


Figura 3. Modelo aproximado de las dos llantas del robot

Las configuraciones de robot diferencial permiten los siguientes movimientos para dos llantas:

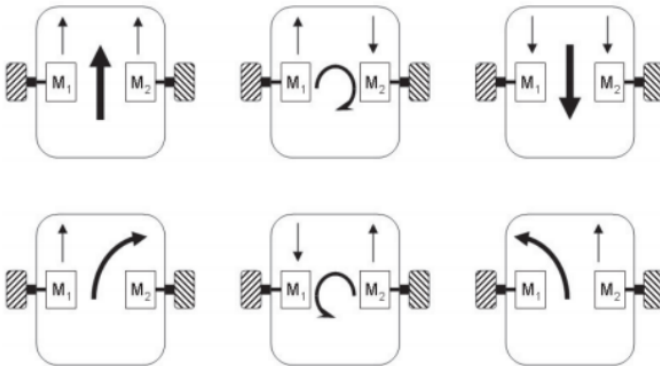


Figura 4. Movimientos permitidos por una configuración diferencial de dos llantas

Sin embargo la tercera llanta es una rueda estándar fija, cuyo ángulo de plano con respecto al chasis es fijo, por lo tanto tiene una restricción de rodadura, lo cual no permite los movimientos 2 y 5 de rotación neta, mostrados en el diagrama de movimientos diferenciales.

I-B. Framework y ambiente de simulación

El robot se constituye sobre el framework ROS Kinetic, y en el entorno de simulación Gazebo, dado que representan herramientas confiables para la simulación de robots, y su control [5].

La cinemática del robot puede describirse mediante las ecuaciones (1).

$$\begin{cases} \dot{x} = v \cos \theta, \\ \dot{y} = v \sin \theta, \\ \dot{\theta} = \omega; \end{cases} \quad (1)$$

I-C. Control de Movimiento

A partir de la cinemática inversa del robot (ec. 2), se puede calcular la velocidad de cada rueda a partir de un vector de velocidad lineal y angular dado.

$$\begin{bmatrix} \omega_r \\ \omega_l \end{bmatrix} = \frac{1}{R} \begin{bmatrix} \cos(\phi) & \sin(\phi) & L \\ \cos(\phi) & \sin(\phi) & -L \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} \quad (2)$$

A partir de aquí se adopta una estrategia de control no holonómico [6] debido a la restricción e manipulabilidad impuesta por la estructura del robot (ver figura 5). Se busca suavizar el movimiento del robot generando una diferencia de velocidad en las que permita al robot girar y corregir su trayectoria sin dejar de avanzar. Para el control de bajo nivel se puede hacer uso de un controlador PID.

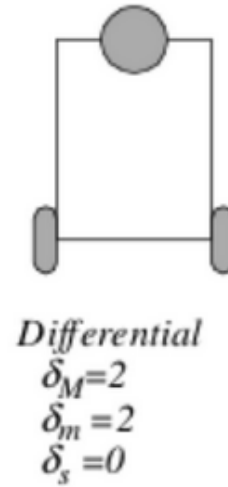


Figura 5. Modelo diferencial con sus grados de maniobrabilidad.

Sin embargo, uno de los problemas que surgen al adoptar esta estrategia de control es la incertidumbre aportada por los sensores utilizados en el lazo de realimentación, la cual afecta la lectura de la posición y de la velocidad en un instante dado. El uso de sensores como encoders implica la introducción de ruido en altas frecuencias, y la información de

odometría aportada por los mismos usualmente está sujeta a *drift*, así como muchos fenómenos de la dinámica del sistema que pueden no estar modelados [7]. Al carecer de un sensor que permita leer directamente la información de la posición del robot, hace falta implementar observadores que permitan estimar el valor de posición y velocidad a partir de los sensores disponibles. Una de las estrategias más utilizadas en robótica es la adaptación de un filtro extendido de Kalman que permite encontrar el estado estimado óptimo presente del sistema a partir del análisis probabilístico de sus estados pasados [8].

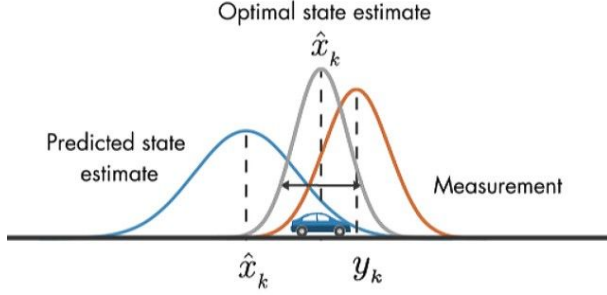


Figura 6. El filtro de Kalman extendido estudia las diferentes distribuciones de probabilidad de medición aportadas por los sensores del robot.

I-D. Accesibilidad

Para un robot ayudante una de las principales cualidades debe ser la accesibilidad, es imposible ayudar a un operario si el robot se encuentra fuera de su alcance, como solución sencilla y autónoma el robot debe seguir a su operario asignado a donde quiera que este vaya, para esto planteamos dos opciones principales:

- Seguimiento de persona: el robot detecta a través de la cámara una persona designada, para seguirla durante el tiempo de trabajo en la
- Seguimiento de objeto: El robot detectara un objeto indicador en la ropa del obrero y seguirá este a través de la planta

Para el modo de seguimiento del robot fue necesario decidirse por el método de seguimiento de objeto porque:

- Al haber muchos operarios operarios en una fabrica el robot puede confundir dos operarios y comenzar a seguir a uno erróneo
- Con los distintos objetos de identificación por prenda del usuario el robot solo seguirá a un mismo operario por toda la fabrica
- Para identificación de coordenadas objetos vistosos son mas fácil de identificar y de aproximar a una sola coordenada, mientras que las personas son mas grandes y con prendas que pueden ser confundidas con el fondo, dificultando la identificación

I-E. Visual SLAM

Visual SLAM (simultaneous localization and mapping) es el proceso mediante el cual se determina la posición y orientación

de una cámara con respecto a su entorno, mientras que simultáneamente se mapea el entorno alrededor de dicha cámara. A pesar de que existen múltiples métodos para implementar SLAM, muchos de ellos involucran sensores complejos como LiDAR y cámaras infrarrojas (como Kinect). En ese orden de ideas, en [9] se propone un marco de trabajo para hacer uso de las cámaras digitales disponibles en un smartphone, lo cual supone una ventaja a la hora de implementar sistemas robóticos móviles costo-eficientes. En dicho trabajo se hace uso de ORB-2 Monocular [10] [11], una alternativa open-source a SIFT y SURF, implementada en OpenCV y fácilmente implementable en ROS [12] [13] [14].

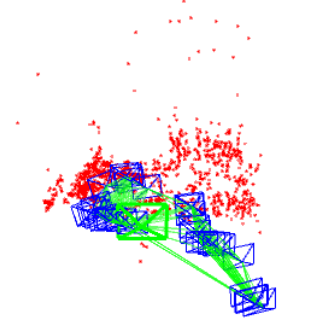


Figura 7. Nube de puntos captada utilizando ORB-2 monocular con ROS Kinetic. Tomado de [14]

II. PROCEDIMIENTOS Y MÉTODOS

II-A. Construcción del modelo en ROS

II-A1. Estructura del Astromech: El sistema físico consiste en una armadura tripodal apoyada en dos llantas de tracción. Se modela el robot a partir de un archivo `.urdf` para poder manipularlo mediante `roscntrol` y evitar problemas de compatibilidad con ROS. Las geometrías se simplifican y se establece asume el centro de gravedad en el centro de la caja naranja de la figura 8. Los parámetros que definen el modelo cinemático del robot se resumen en la tabla I.

Cuadro I
PARÁMETROS QUE DESCRIBEN EL MODELO CINEMÁTICO DEL ROBOT.

Parámetros	Unidades
l	120.7 mm
r	13.85 mm
α_1	$\pi/2$ rad
α_2	$-\pi/2$ rad
β_1	0 rad
β_2	π rad

Las velocidades máximas se establecen en $v = 1 \text{ m s}^{-1}$ y $\omega = \pi \text{ rad s}^{-1}$

II-A2. Control en lazo cerrado: La implementación de control de lazo cerrado se limita al uso de una cámara únicamente en el modelo real, sin embargo el modelo simulado puede utilizar la odometría suministrada por el `topic odom` para lo que se programó.

II-B. Implementación física

Teniendo en cuenta el mínimo uso de recursos la implementación física pasó por diferentes etapas, primero se verificó la viabilidad del control de motores, primero usando un Arduino Leonardo como *slave*, después se hizo un diseño con transistores 2N2222A, y finalmente se implementó un IC L298N, esto se hizo usando motores de 3V reciclados. La Raspberry está alimentada con un cargador de 5V, 2A, y el integrado tiene un V_{ss} de 9V que los provee una batería. El driver se conecta a los pines GPIO 24,23,25 y 17,22,27 para los motores derecho e izquierdo respectivamente. Posteriormente se ensambló en la estructura y se sintonizaron los valores PWM para tener velocidades de giro similares. El control de estas velocidades se realizó con un ciclo que posee tres velocidades diferentes y tiene 5 posibilidades, avance, retroceso, giro a la izquierda, giro a la derecha y stop. Esto se implementó en Python con las librerías RPi.GPIO.

Considerando que el control servo-visual analizado anteriormente envía sus comandos al topico *cmd_vel* se puede implementar con el robot, sin embargo dada la capacidad computacional de la Raspberry utilizada no se implementó el control visual en este.

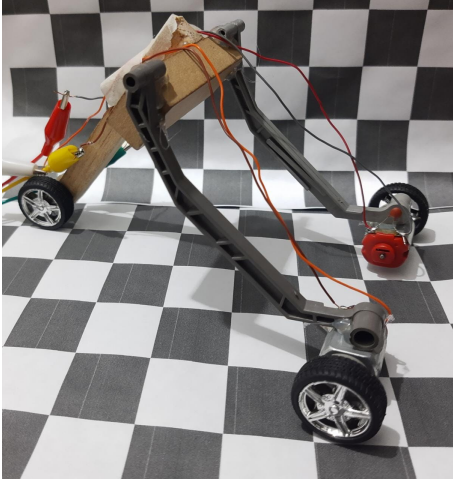


Figura 10. Construcción del prototipo.

II-C. Implementación de ROS en el modelo Físico

Posteriormente a la validación de la implementación viable en los motores se configuró un nodo de ROS para el control de los motores, se probó diferentes metodologías, primero adaptando el nodo al código usado anteriormente del control, pero no fue posible que se comunicara correctamente, se indagó y la solución fue la adaptación del controlador *teleop_twist_keyboard* puesto que utiliza como librería *termios.h* que es una interfaz que se provee para el control de puertos de comunicación asíncrona, hecho esto es efectivo el movimiento de las ruedas de acuerdo al comando que publica el nodo de este controlador.

III. RESULTADOS Y ANÁLISIS

III-A. Simulación

Durante la construcción se encontraron diversas dificultades, durante la construcción del modelo *urdf* se encontró que los valores de inercia calculados para el modelo tuvieron que ser modificados sensiblemente y sintonizados de manera empírica hasta lograr un ajuste que provocara un comportamiento relativamente estable en el sistema. No es posible utilizar exactamente el mismo código de simulación que el real, puesto que las librerías RPi.GPIO, necesarias para el control de los pines GPIO de la Raspberry sólo están disponibles para Raspberry, por ende no se puede ejecutar el simulador con esta librería en un computador, y la Raspberry no tiene el poder computacional para correr Gazebo.

A continuación se mostrará los pasos implementados en el simulador usando un robot *Turtlebot* que utiliza los mismos comandos de velocidad.

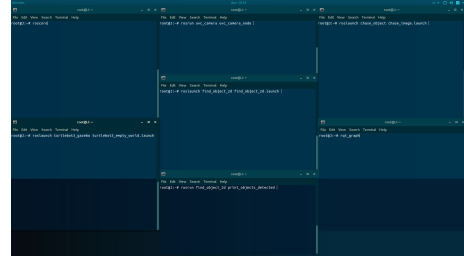


Figura 11. Ventanas requeridas.

Primero se preparan todas las diferentes ventanas que se van a implementar, las cuales se corren de izquierda a derecha, superior a inferior, comenzando por el *master node* de ROS.

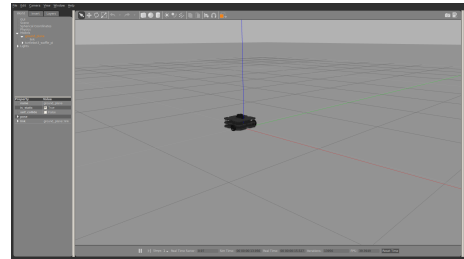


Figura 12. Ejecución de *roscore*.

Posteriormente iniciaremos el entorno *Gazebo* del robot que sea de nuestra preferencia, en este caso se usó *Turtlebot3*. Seguidamente iniciamos los drivers de la cámara, que en nuestro caso corresponden a *uvc_camera*.

A continuación corremos el archivo launch de nuestro paquete *find_object_2d* para iniciar *Find - Object* con las imágenes precargadas, cuyas imágenes de referencia corresponden a:

Después procedemos a lanzar el nodo que publica las posiciones de nuestro objeto en */imageLocation*.

Y luego ejecutamos el nodo que nos enviará las direcciones a nuestro robot.

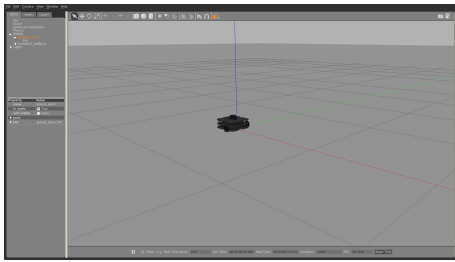


Figura 13. Ejecución del entorno de simulación.

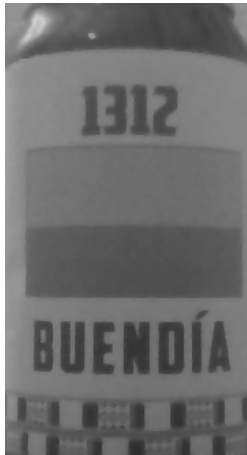


Figura 14. Objeto a buscar en la identificación.

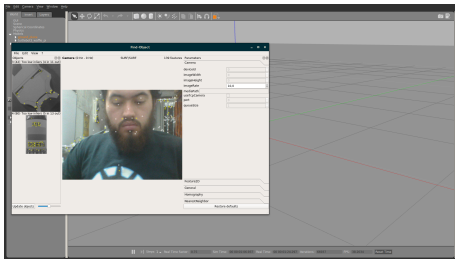


Figura 15. Lanzamiento del reconocimiento visual.

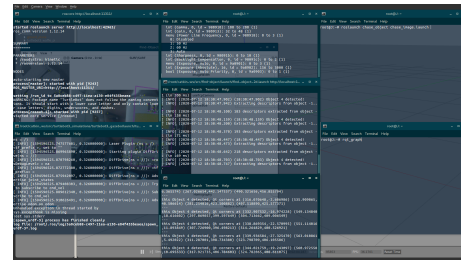


Figura 17. Nodo para comandos *tTwist*.

Finalmente observamos como el robot se mueve dependiendo de si encuentra el objeto que se solicitó.

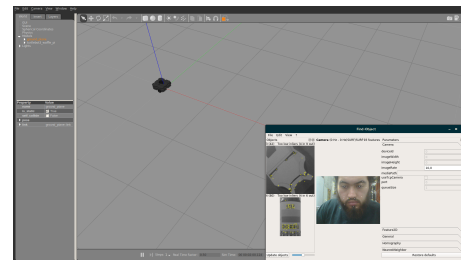


Figura 18. Robot detenido.

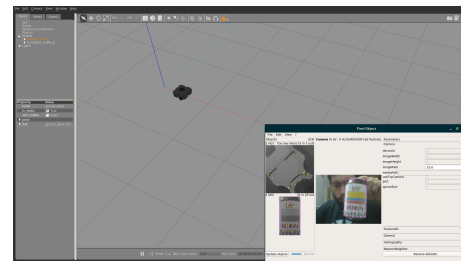


Figura 19. Robot en movimiento dado que encuentra al objeto.

Finalmente podemos observar el árbol *rqf_graph* de nuestro robot.



Figura 20. Árbol *rqt_graph*

III-B. Implementación física

Los motores usados no proveen el suficiente torque para que el robot sea móvil con la alimentación actual. Se logró obtener el control de los motores en lazo abierto, utilizando un único nodo para interpretar el teclado como direcciones de mando, y este a su vez puede comunicarse con el modelo simulado.

De igual manera resulta necesario tener una fuente de alimentación externa, lo ideal son baterías Li-Po, sin embargo una batería de 9V comercial es suficiente para la activación de los motores. Aunque se realizaron pruebas con alimentación de adaptadores de pared de 9V y 0.8 mA, se obtuvieron mejores resultados con la batería.

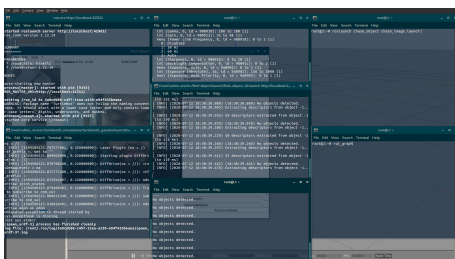


Figura 16. Nodo que envía la posición de las imágenes.

IV. CONCLUSIONES

- La implementación de ROS requiere de un sistema con compatibilidad con Ubuntu o Lubuntu como mínimo, para poder ejecutarlo sin necesidad de adaptar todas las librerías bases del framework, afortunadamente la Raspberry Pi 3B cuenta con soporte de Ubiquity Robotics.
- La implementación en nodos de ROS aunque permite separar diferentes interacciones del sistema, también puede saturarse de nodos con la intención de simplificar lectura del funcionamiento del sistema.
- Los parámetros de inercia en los robots simulados en Gazebo no se ajustan totalmente a la realidad, esto se evidencia en casos como el Kobuki, donde sus valores son calculados empíricamente como lo indica el fabricante.
- Se puede separar en nodos diferentes la implementación real y la simulada separando el uso de la interfaz de GPIO de la Raspberry con un nodo que escuche el tópico `cmd_vel`, aunque esto puede dilatar la estructura compacta del código planteado.

REFERENCIAS

- [1] M. Hengstler, E. Enkel, and S. Duelli, "Applied artificial intelligence and trust—the case of autonomous vehicles and medical assistance devices," *Technological Forecasting and Social Change*, vol. 105, pp. 105–120, 2016.
- [2] Audi of America, "Audi deploys innovative communications robots to streamline vehicle service visits." [en línea] <https://www.audiusa.com/newsroom/news/press-releases/2014/06/audi-deploys-innovative-communications-robots-streamline-vehicle-service>, June 2014.
- [3] P. Gareffa, "Audi equips dealerships with robotic telepresence mechanics." [en línea] <https://www.edmunds.com/car-news/audi-equips-dealerships-with-robotic-telepresence-mechanics.html>, June 2014.
- [4] N. Chivarov, D. Chikurtev, K. Yovchev, and S. Shivarov, "Cost-oriented mobile robot assistant for disabled care," *IFAC-PapersOnLine*, vol. 48, no. 24, pp. 128–133, 2015.
- [5] K. Takaya, T. Asai, V. Kroumov, and F. Smarandache, "Simulation environment for mobile robots testing using ros and gazebo," in *2016 20th International Conference on System Theory, Control and Computing (ICSTCC)*, pp. 96–101, IEEE, 2016.
- [6] L. E. S. Guzmán, M. A. M. Villa, and E. L. R. Vásquez, "Seguimiento de trayectorias con un robot móvil de configuración diferencial," *Ingenierías USBMed*, vol. 5, no. 1, pp. 26–34, 2014.
- [7] K. Yoshida, "The spacedyn: a matlab toolbox for space and mobile robots," in *Proceedings 1999 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human and Environment Friendly Robots with High Intelligence and Emotional Quotients (Cat. No. 99CH36289)*, vol. 3, pp. 1633–1638, IEEE, 1999.
- [8] S. Karakaya, G. Kucukyildiz, and H. Ocak, "A new mobile robot toolbox for matlab," *Journal of Intelligent & Robotic Systems*, vol. 87, no. 1, pp. 125–140, 2017.
- [9] W.-W. Kao and B. Q. Huy, "Indoor navigation with smartphone-based visual slam and bluetooth-connected wheel-robot," in *2013 CACS International Automatic Control Conference (CACS)*, pp. 395–400, IEEE, 2013.
- [10] R. Mur-Artal and J. D. Tardós, "Orb-slam: Tracking and mapping recognizable," in *Workshop on Multi View Geometry in Robotics (MVIGRO)-RSS*, 2014.
- [11] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [12] M. Rojas-Fernández, D. Mújica-Vargas, M. Matuz-Cruz, and D. López-Borreguero, "Performance comparison of 2d slam techniques available in ros using a differential drive robot," in *2018 International Conference on Electronics, Communications and Computers (CONIELECOMP)*, pp. 50–58, IEEE, 2018.
- [13] B. Williams, M. Cummins, J. Neira, P. Newman, I. Reid, and J. Tardós, "A comparison of loop closing techniques in monocular slam," *Robotics and Autonomous Systems*, vol. 57, no. 12, pp. 1188–1197, 2009.
- [14] J. Zijlmans, "Orb-slam2: Implementation on my ubuntu 16.04 with ros kinect." [en línea] <https://medium.com/@j.zijlmans/orb-slam-2052515bd84c>, 2017.